

Week 5. Cilium & eBPF

Cilium이란?

Cilium은 eBPF 기반의 오픈소스 CNI이다. Pod 간 네트워킹뿐 아니라 로드 밸런싱, 네트워크 정책, 관측성까지 함께 제공한다. 패킷 처리 로직이 Linux 커널 안에서 eBPF로 동작하기 때문에, 애플리케이션 코드나 컨테이너 설정을 건드리지 않고도 정책을 적용하고 바꿀 수 있다.

등장 배경

기존 방식에는 두 가지 확장성 문제가 있었다.

첫째는 iptables다. kube-proxy는 Service마다 iptables 규칙을 만들고, iptables는 규칙을 순서대로 확인한다. Service가 많아질수록 규칙이 늘어나고 탐색·갱신 비용도 커진다.

둘째는 IP 기반 정책이다. 방화벽이 출발지 IP로 트래픽을 거르면, Pod가 생길 때마다 모든 노드의 규칙을 고쳐야 한다. Pod IP는 수시로 바뀌기 때문에 정책 기준으로 쓰기에 불안정하다.

Cilium은 규칙 목록 대신 eBPF map(해시 테이블)을 쓰고, 정책 기준을 IP에서 label 기반 identity로 바뀌어서 이 문제를 풀었다.

주요 기능

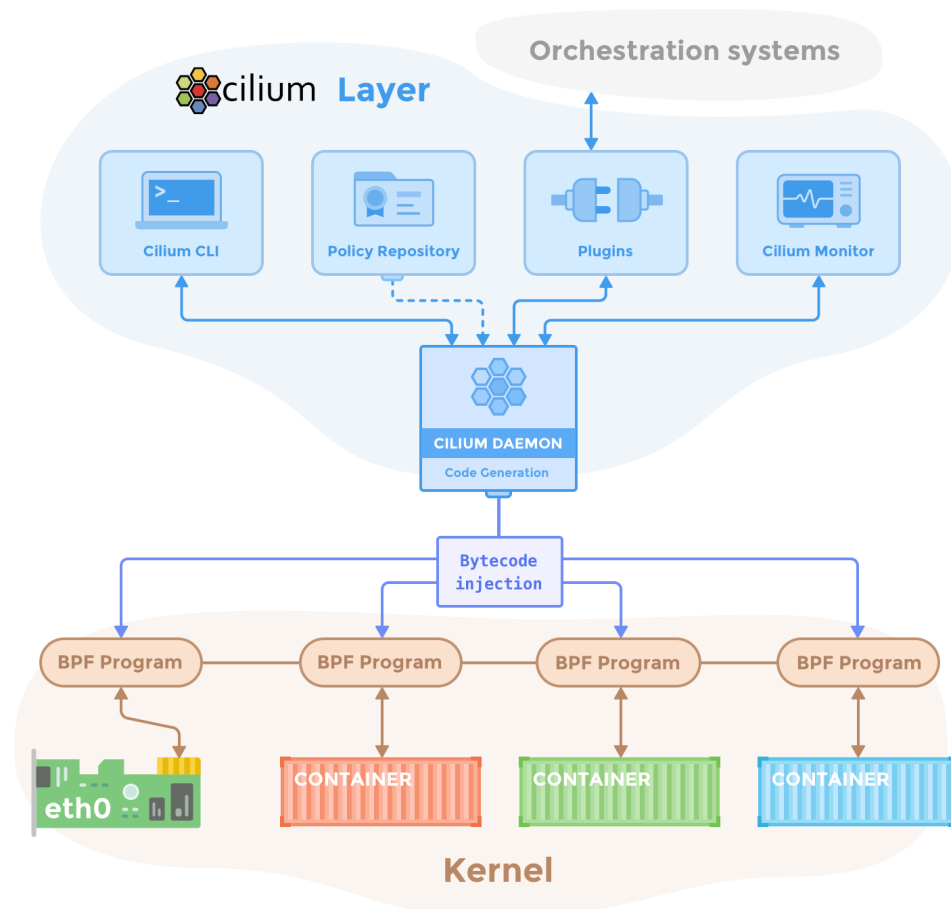
- CNI: VXLAN/Geneve 기반 Overlay와 Native routing을 모두 지원한다. BGP 등으로 경로 광고도 할 수 있다.
- 로드 밸런싱: Service 처리를 eBPF로 구현해 kube-proxy를 대체할 수 있다.
- 네트워크 정책: identity 기반으로 L3~L7 정책을 적용한다. 도메인 이름(FQDN) 기반 정책도 지원한다.
- 관측성: Hubble로 Pod 간 flow와 drop 사유를 확인할 수 있다.
- 그 외: 여러 클러스터를 연결하는 Cluster Mesh, 암호화와 Gateway API를 지원하는 Service Mesh 기능도 있다.

기존 CNI와 다른 점

Week 4에서 본 Flannel은 VXLAN overlay로 Pod 간 연결을 만드는 데 집중하고, NetworkPolicy는 직접 처리하지 않는다. Calico는 iptables 기반 데이터 플레인과 eBPF 데이터 플레인 중 하나를 선택할 수 있다.

Cilium은 처음부터 eBPF를 데이터 플레인으로 쓴다. 패킷은 iptables 체인이 아니라 커널 hook에 붙은 eBPF 프로그램에서 처리되고, 정책은 IP가 아니라 identity로 판단한다. 또 CNI, kube-proxy 대체, 정책, 관측성이 모두 같은 eBPF datapath 위에서 동작하기 때문에, 정책 판정 결과를 Hubble에서 그대로 볼 수 있다.

Cilium Architecture



구성요소

구성요소	실행 위치	역할
cilium-agent	각 노드 (DaemonSet)	K8s 이벤트를 감시하고, 네트워크를 제어하는 eBPF 프로그램을 관리한다
cilium-operator	클러스터 단위	IPAM처럼 클러스터 전체에서 한 번만 하면 되는 작업을 맡는다
cilium-cni	각 노드	Pod 생성·삭제 시 호출되어 agent에 datapath 설정을 요청한다

구성요소	실행 위치	역할
Hubble	agent 내장 + Relay	노드별 flow를 수집하고 클러스터 전체로 모아 보여준다

Operator는 패킷 전달이나 정책 판정 경로에 있지 않다. 그래서 operator가 잠시 내려가도 기존 트래픽은 계속 처리되고, 새 Pod의 IP 할당 정도가 늦어질 수 있다.

Control Plane과 Data Plane

cilium-agent는 K8s API를 감시하면서 정책, Service, identity 정보를 계산해 eBPF map에 기록한다. 실제 패킷은 커널에 로드된 eBPF 프로그램이 이 map을 조회하면서 처리한다. 즉 agent는 설정하는 쪽이고, 패킷 처리는 커널 안의 eBPF가 한다.

eBPF 기본 개념 및 Hook

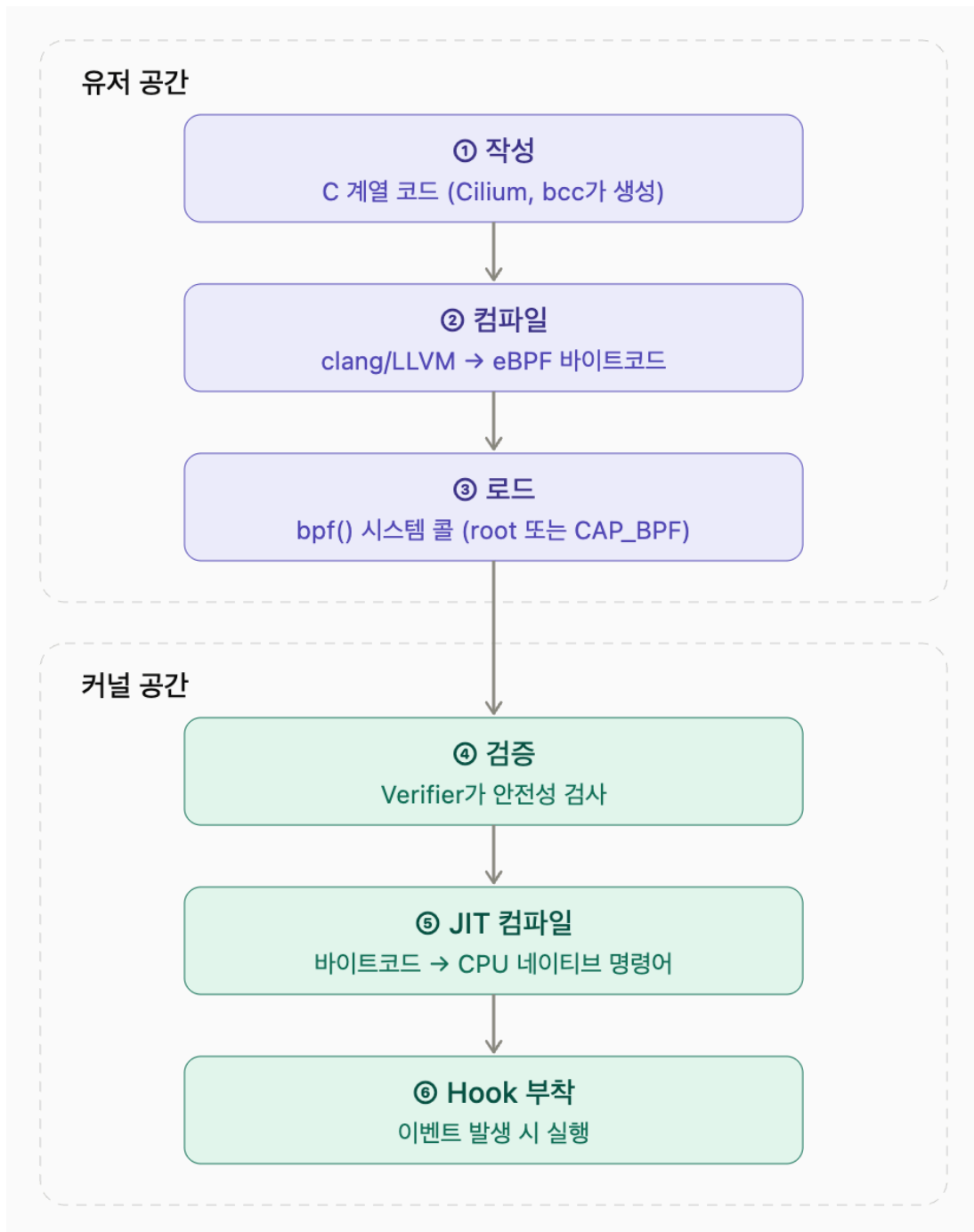
eBPF란?

eBPF는 커널 소스를 고치거나 커널 모듈을 올리지 않고도, 커널 안에서 샌드박스 프로그램을 실행할 수 있게 해주는 기술이다.

커널은 시스템 전체를 볼 수 있어서 네트워킹이나 보안 기능을 넣기 좋은 위치지만, 안정성이 중요해서 쉽게 바뀌지 않는다. 새 기능을 넣으려면 커널 커뮤니티를 설득해 몇 년을 기다리거나, 커널을 망가뜨릴 위험이 있는 커널 모듈을 써야 했다. eBPF는 그 사이에서 런타임에 안전하게 커널 동작을 바꿀 수 있는 방법을 제공한다.

이름의 BPF는 원래 Berkeley Packet Filter로, tcpdump 같은 도구에서 패킷을 거르는 용도였다. 이후 자료구조와 패킷 조작 기능이 추가되면서 extended BPF(eBPF)가 되었고, 지금은 패킷 필터링 외에도 다양하게 쓰여 약어의 의미는 크게 남아 있지 않다. 예전 BPF는 cBPF(classic BPF)라고 구분해서 부른다.

동작 흐름



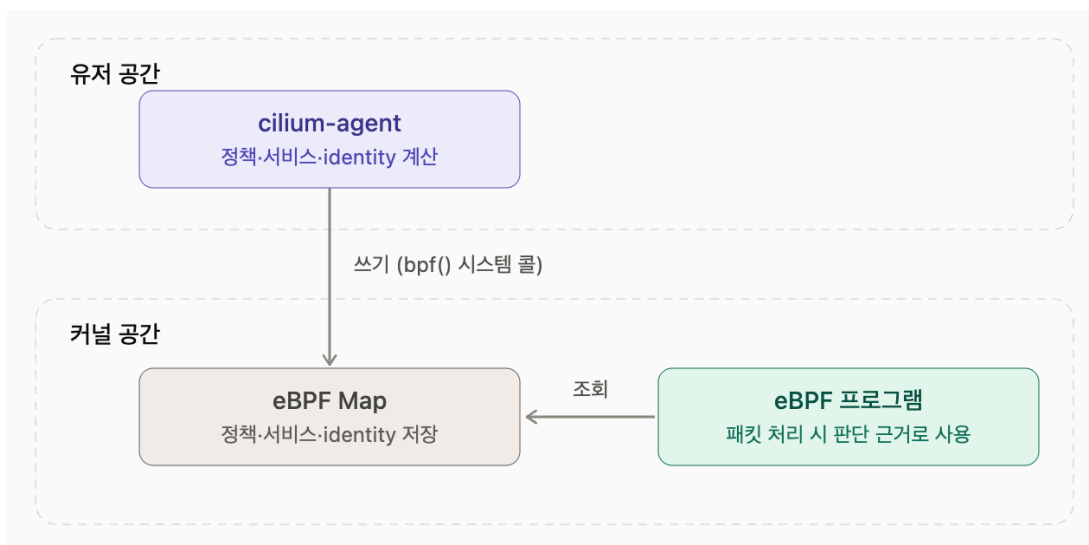
프로그램이 커널에 로드되면 Verifier가 먼저 안전성을 검사한다. 프로그램이 반드시 종료되는지, 범위를 벗어난 메모리에 접근하지 않는지, 크기와 복잡도가 제한 안에 있는지를 본다. Verifier는 안전하게 실행될 수 있는지만 확인할 뿐, 프로그램이 무슨 일을 하는지는 검사하지 않는다. 검증을 통과하면 JIT 컴파일로 CPU 명령어로 바뀌어 hook에 붙는다.

eBPF Map

eBPF 프로그램이 상태를 저장하고 정보를 주고받는 자료구조이다. 해시 테이블, 배열, LRU, LPM 등 여러 형태가 있고, 커널의 eBPF 프로그램과 유저 공간의 애플리케이션 양쪽

에서 접근할 수 있다.

Cilium에서는 agent가 정책·서비스·identity 정보를 map에 쓰고, eBPF 프로그램이 패킷을 처리할 때 이 map을 조회한다.



eBPF가 네트워크 처리에 들어가는 위치

eBPF 프로그램은 이벤트 기반으로, 커널이 특정 hook 지점을 지날 때 실행된다. Cilium이 주로 쓰는 hook은 다음과 같다.

Hook	위치	Cilium에서의 용도
XDP	NIC 드라이버, 패킷 수신 직후 가장 이른 지점	불필요한 패킷 drop, NodePort 로드 밸런싱 가속
tc ingress/egress	네트워크 인터페이스(veth 포함), L3 처리 이전	Pod 정책 적용, 트래픽 전달
Socket (cgroup)	<code>connect()</code> , <code>sendmsg()</code> 같은 시스템 콜	소켓 레벨 로드 밸런싱 (kube-proxy 대체)

Week 1에서 본 Netfilter hook과 비교하면, XDP와 tc는 iptables 규칙보다 앞에서 실행되고, socket hook은 패킷이 만들어지기도 전에 실행된다. 그래서 Cilium은 iptables 체인에 도달하기 전에 패킷을 처리할 수 있다.

Cilium Datapath

Pod 트래픽은 어디서 처리되는가

Pod는 veth pair로 호스트와 연결되고, 호스트 쪽 인터페이스는 `lxc*` 라는 이름을 가진다. Cilium은 이 호스트 쪽 veth의 tc ingress hook에 eBPF 프로그램을 붙여 Pod에서 나가는 트래픽을 감시하고 정책을 적용한다.

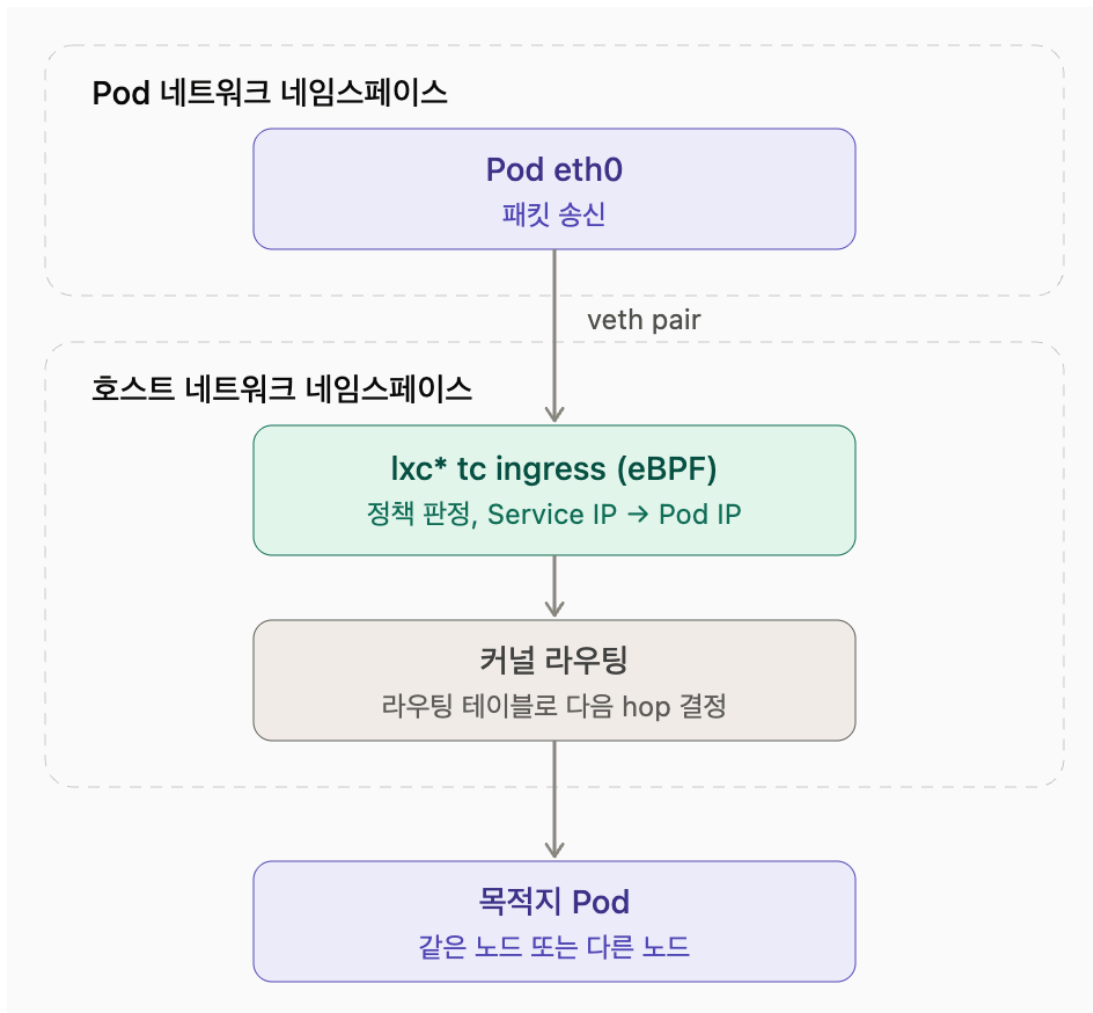
Pod가 보내는 트래픽을 ingress hook에서 처리한다는 점이 처음엔 헷갈렸는데, Pod의 `eth0` 에서 나간 패킷은 호스트 쪽 `lxc*` 입장에서는 들어오는 패킷이기 때문이다.

Datapath 구성 요소

Cilium은 hook별 eBPF 프로그램을 조합해서 몇 가지 역할 단위를 만든다.

- Prefilter (XDP): 가장 앞단에서 유효하지 않은 목적지로 가는 패킷을 걸러낸다.
- Endpoint Policy (tc): 패킷의 identity를 찾아 정책에 따라 drop하거나 전달 경로를 정한다. L3/L4 정책은 여기서 적용된다.
- Service (tc): 목적지 IP와 포트로 map을 조회해 backend로 보낸다.
- L7 Policy: L7 정책이 필요한 트래픽은 Envoy 프록시로 넘긴다.
정책 판정도, Service 로드 밸런싱도 규칙을 순서대로 비교하는 게 아니라 map 조회로 처리된다.

패킷 흐름



Pod에서 나간 패킷은 `lxc*`의 tc ingress에서 정책 판정을 받고, 이후 커널 라우팅 테이블에 따라 목적지로 전달된다.

Service로 보내는 트래픽의 경우, Socket-LB가 켜져 있으면 `connect()` 시점에 이미 Service IP가 backend Pod IP로 바뀌어 있다. Socket-LB를 쓰지 않는 환경에서는 tc 단계의 Service 처리에서 주소가 바뀐다.

eBPF Map

agent가 계산한 정보는 map에 저장되고, eBPF 프로그램은 패킷을 처리할 때 이를 조회한다. connection tracking 테이블도 iptables가 아닌 Cilium 자체 map으로 관리한다. 모든 map은 최대 크기가 정해져 있어서, 대규모 클러스터에서는 map 크기를 미리 고려해야 한다.

iptables를 아예 안 쓰는건가?



커널 버전에 따라 eBPF로 구현할 수 있는 기능이 달라서, 지원되지 않는 기능은 기존 iptables 방식으로 처리한다. Cilium이 없앤 것은 kube-proxy가 만들던 대량의 Service 규칙이지, iptables 자체는 아니다.

라우팅 모드

	Encapsulation	Native routing
방식	노드 간 VXLAN/Geneve 터널	캡슐화 없이 네트워크 라우팅 사용
조건	노드끼리 통신만 되면 됨	네트워크가 Pod IP를 라우팅할 수 있어야 함
단점	터널 헤더만큼 MTU 감소	인프라 쪽 라우팅 설정 필요

별도 설정이 없으면 Encapsulation 모드로 동작한다.

kube-proxy Replacement

기존 kube-proxy 방식

kube-proxy는 각 노드에서 API 서버의 Service·Endpoint 변경을 감시하고, 변경이 생기면 iptables 규칙을 추가하거나 삭제한다. Pod에서 Service IP로 보낸 트래픽은 이 규칙을 거쳐 실제 Pod로 전달된다.

iptables는 규칙을 위에서부터 순서대로 확인하기 때문에, Service와 Pod가 늘어나면 규칙도 같이 늘어나고 처리·갱신 비용이 커진다.

Cilium 방식

Cilium도 Service·Endpoint 변경을 감시하는 건 같다. 다만 iptables 규칙 대신 노드의 eBPF map을 갱신한다. map은 해시 테이블이라 Service가 많아져도 조회 한 번으로 backend를 찾을 수 있다.

ClusterIP, NodePort, LoadBalancer, externalIPs 타입을 모두 처리할 수 있고, hostPort도 지원해서 portmap 플러그인을 따로 쓸 필요가 없다.

Socket-LB

클러스터 내부 트래픽은 Socket-LB로 처리한다. 애플리케이션이 `connect()` 를 호출하는 순간 목적지가 Service IP인지 확인하고, 그 자리에서 backend를 골라 연결 대상을 바꾼다. 애플리케이션은 Service에 연결했다고 생각하지만 실제 소켓은 backend Pod에 연결되어 있다. 연결 시점에 한 번만 바꾸기 때문에 이후 패킷마다 주소를 변환할 필요가 없다.

클러스터 외부에서 들어오는 트래픽은 별도로 처리되며, XDP 가속, DSR, Maglev 해싱 같은 L4 로드 밸런싱 기능을 쓸 수 있다.

NetworkPolicy / Identity

IP 기반 정책의 한계

일반적인 방화벽은 출발지 IP와 목적지 포트로 트래픽을 거른다. 그런데 Kubernetes에서는 Pod가 수시로 생성·삭제되면서 IP가 계속 바뀐다. IP로 정책을 관리하면 Pod가 새로 뜰 때마다 모든 노드의 규칙을 고쳐야 하고, 클러스터가 커질수록 부담이 된다.

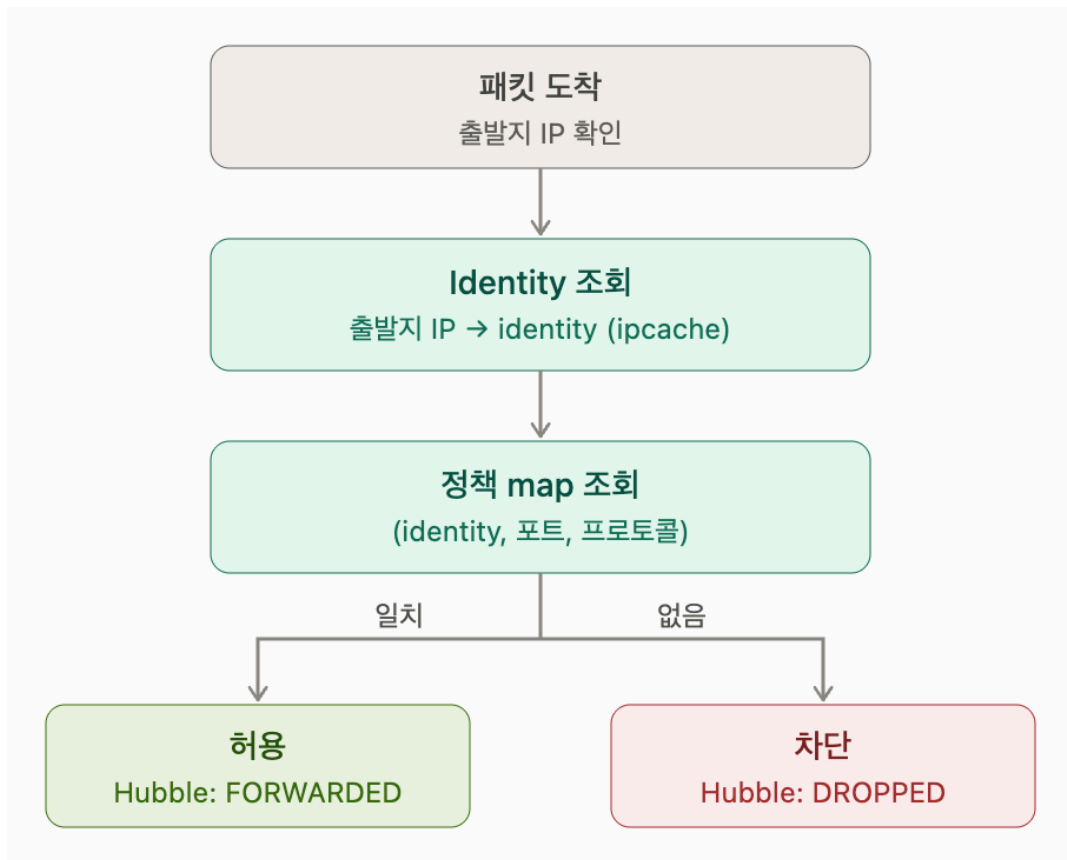
Identity

Cilium은 정책 기준을 IP에서 label로 바꿨다. 같은 label 조합을 가진 Pod들은 하나의 숫자 identity를 공유한다. 예를 들어 `app=frontend` Pod가 10개 떠 있어도 identity는 하나다.

Pod가 늘거나 줄어도 identity는 그대로이기 때문에 정책 규칙은 바꿀 필요가 없다. 새 Pod의 IP와 identity의 매핑만 각 노드에 갱신하면 된다.

Cilium이 관리하지 않는 대상도 정책에 넣을 수 있도록 `reserved:` 접두사가 붙은 특수 identity가 있다. 로컬 노드를 뜻하는 `reserved:host`, 클러스터 바깥을 뜻하는 `reserved:world`, 다른 노드들을 뜻하는 `reserved:remote-node` 등이 있다.

정책 판정 흐름



정책 판정은 datapath의 Endpoint Policy 단계에서 이뤄진다. endpoint마다 정책 map이 있고, 여기에 허용된 identity·포트·프로토콜 조합이 저장된다. iptables처럼 규칙을 하나씩 비교하는 게 아니라 map 조회 한 번으로 판정이 끝난다.

정책 종류

	Kubernetes NetworkPolicy	CiliumNetworkPolicy
적용 계층	L3/L4	L3/L4 + L7
추가 기능	-	DNS(FQDN) 기반 정책
적용 범위	namespace	namespace, 클러스터 전역(CCNP)

L7 정책을 쓰면 HTTP 메서드나 경로 단위까지 제어할 수 있다. 예를 들어 `/public/` 경로에 대한 GET 요청만 허용하는 식이다. DNS 기반 정책은 외부 API처럼 IP가 자주 바뀌는 대상으로 나가는 트래픽을 도메인 이름으로 제어할 때 쓸 수 있다.

Hubble Overview

Hubble이란

Hubble은 Cilium과 eBPF 위에서 동작하는 네트워크·보안 관측 도구이다. Pod에 별도 에이전트를 붙이지 않고, Cilium datapath에서 발생하는 이벤트를 그대로 가져와 보여준다.

구성

각 노드의 cilium-agent 안에 Hubble Server가 내장되어 해당 노드의 flow를 제공한다. Hubble Relay는 모든 노드의 Server를 모아 클러스터 전체를 하나의 API로 보여주고, CLI와 UI는 Relay에 연결해 flow를 조회하거나 서비스 의존성 맵을 그려준다.

무엇을 볼 수 있나

공식 문서는 Hubble로 답할 수 있는 질문을 네 가지로 나눈다.

- 서비스 의존성: 어떤 서비스끼리 통신하는지, 어떤 HTTP 호출이 오가는지
 - 네트워크 모니터링: 통신이 실패했다면 DNS 문제인지, L4와 L7 중 어디서 끊겼는지
 - 애플리케이션 모니터링: 4xx·5xx 응답 비율, 서비스 간 지연 시간
 - 보안 관측: 네트워크 정책으로 차단된 연결, 클러스터 외부에서 들어온 접근
- flow는 IP 대신 Pod 이름, namespace, label로 표시된다. identity 기반 구조이기 때문에 가능한 부분이다.

관찰과 정책은 어떻게 연결되나

정책 판정과 flow 관찰은 같은 eBPF datapath에서 일어난다. 패킷이 Endpoint Policy 단계에서 허용되거나 차단되면 그 결과가 이벤트로 남고, Hubble은 이를 flow로 보여준다. 그래서 flow마다 **FORWARDED** 나 **DROPPED** 같은 판정 결과와 drop 사유가 함께 찍힌다.

덕분에 정책도 관찰에서 시작해서 만들 수 있다. Hubble로 실제 통신 흐름을 먼저 보고, 필요한 통신만 허용하는 정책을 만든 다음, 의도하지 않은 통신이 **DROPPED** 로 찍히는지 다시 확인하면 된다.

▼ 참고자료

<https://docs.cilium.io/en/stable/overview/intro/>

<https://ebpf.io/ko-kr/what-is-ebpf/>

<https://docs.cilium.io/en/stable/overview/component-overview/>

<https://docs.cilium.io/en/stable/network/ebpf/intro/>

질문

▼ 1. cilium-agent가 재시작되는 동안에도 기존 Pod 간 트래픽은 정상적으로 처리될까?



cilium-agent는 Control Plane 역할을 하고, 실제 패킷 처리는 커널에 로드된 eBPF 프로그램과 map이 담당한다. map은 BPF 파일시스템(bpffs)에 고정되어 있어 agent가 재시작되어도 identity, Service, connection tracking 정보가 남아 있고, 기존 트래픽은 이를 이용해 계속 처리된다.

다만 agent가 없는 동안에는 새 Pod의 identity 할당이나 정책 변경이 반영되지 않는다. agent가 복구되면 API 서버의 상태를 다시 읽어 map을 갱신한다. 또 버전에 따라 agent 시작 직후 eBPF 프로그램이 잠깐 빠지는 문제(1.13.0)가 있었던 것처럼, 항상 완전히 무중단이라고 보기는 어렵다.

▼ 2. 라우팅 모드에 따라 받는 노드는 출발지 identity를 어떻게 알아내는가?

라우팅 모드에 따라 출발지 identity를 알아내는 방식이 다르다.

Encapsulation 모드에서는 VXLAN이나 Geneve로 패킷을 감쌀 때 터널 헤더에 출발지 identity를 함께 실어 보낸다. 받는 노드는 터널을 해제하면서 헤더의 identity를 바로 정책 판정에 사용하므로, identity 조회를 한 번 줄일 수 있다.

Native routing 모드에서는 캡슐화가 없어 identity를 실을 곳이 없다. 대신 각 노드의 agent가 Kubernetes API를 감시하며 클러스터 전체의 IP와 identity 매핑(ipcache)을 유지한다. 받는 노드는 패킷의 출발지 IP로 ipcache를 조회해 identity를 알아낸다.